

Project 1 — List of Sparse Matrices

A review of intent and functionality

Aren Vista · CMSC 341 — Fall 2026 · 2026-09-14

1	What the project is	1
2	The format itself	2
2.1	The three arrays	2
2.2	The worked example	2
2.3	Reading a member back	6
2.4	Sparsity	7
3	Function-by-function intent	7
3.1	CSR	7
3.2	CSRList	7
4	The invariants everything rests on	8
5	Empty is not all-zero	8
6	Input handling in <code>compress</code>	9
7	Memory, and where it goes wrong	9
8	Order, and the two places it matters	10
9	Error signalling	10
10	Testing	11
11	Suggested order of work	11

1 What the project is

The assignment builds one data structure out of two, each of which solves a different problem, and the interesting part is the seam between them.

The inner structure, `CSR`, is a **compression scheme**. A sparse matrix is one whose members are mostly zero, and storing all $m \times n$ of them wastes memory on values we already know. Compressed Sparse Row throws the zeros away and keeps enough bookkeeping to reconstruct any member on demand.

The outer structure, `CSRList`, is a **singly linked list with a head pointer** whose nodes happen to be `CSR` objects. It is deliberately plain: head insertion, traversal, deep copy, destruction.

The seam is that `CSR` is both a standalone value type (it is copied, compared and assigned as a matrix) and a list node (it carries `m_next`). Almost every memory bug available in this project lives at that seam.

FILE	ROLE	STATUS
<code>csr.h</code>	Interface for both classes. Members, signatures, the two marked spots for private helpers.	given, frozen
<code>csr.cpp</code>	Implementation of both classes. <code>CSR::dump()</code> and <code>CSRList::dump()</code> are supplied to write complete.	

FILE	ROLE	STATUS
<code>driver.cpp</code>	Sample usage. Builds two matrices, copies them, puts them in two lists, prints, then forces an exception.	given
<code>driver.txt</code>	The exact expected output of that driver. The ground truth for <code>compress()</code> and list order.	given
<code>mytest.cpp</code>	The <code>Tester</code> class, its test functions, and <code>main()</code> .	to write

! Important Three files are submitted: `csr.h`, `csr.cpp`, `mytest.cpp`. The class declarations and the provided function bodies may not be modified, no additional libraries may be used, no STL containers inside `CSR` or `CSRList` (test code may use them), and nothing may be written inline in the header. Helper functions are allowed but must be declared in the `private:` sections of `csr.h`.

2 The format itself

2.1 The three arrays

Scan the matrix left to right, row by row, top to bottom, and record only the members that are not zero.

- `m_values` — those non-zero values, in scan order.
- `m_col_index` — the column each of them came from, in the same order. Same length as `m_values`, namely `m_nonzeros`.
- `m_row_index` — offsets into the other two, one per row plus a terminator. Length `m_m + 1`.

`m_row_index[i]` is the cumulative number of non-zeros in rows 0 through $i - 1$. So it starts at 0, never decreases, and its last entry is the total count `m_nonzeros`.

2.2 The worked example

This is the 4×6 matrix from `driver.cpp`, which is also the example in the project description.

$$\begin{bmatrix} 10 & 20 & 0 & 0 & 0 & 0 \\ 0 & 30 & 0 & 40 & 0 & 0 \\ 0 & 0 & 50 & 60 & 70 & 0 \\ 0 & 0 & 0 & 0 & 0 & 80 \end{bmatrix} \quad (2.1)$$

2.2.1 What `compress` is actually handed

Not a matrix — a flat array and two dimensions:

```
int array1[] = {10,20,0,0,0,0,0,30,0,40,0,0,0,50,60,70,0,0,0,0,0,0,80};
aCSR.compress(4, 6, array1, 24);
```

The matrix is a reading of that array, not a property of it. With $n = 6$, member (i, j) lives at flat index $in + j$:

ROW i	FLAT INDICES	MEMBERS
0	0..5	10 20 0 0 0 0
1	6..11	0 30 0 40 0 0
2	12..17	0 0 50 60 70 0
3	18..23	0 0 0 0 0 80

The same 24 numbers with $n = 4$ would be a 6×4 matrix and would compress to something entirely different. Nothing in the array itself says where the rows break.

2.2.2 Pass 1 — count

Walk all $mn = 24$ positions and count the members that are non-zero, without storing anything. Here that gives 8.

That number is `m_nonzeros`, and knowing it up front is what lets the second pass allocate `m_values` and `m_col_index` at exactly 8 entries instead of guessing at the 24-entry worst case.

2.2.3 Pass 2 — fill

Walk the same positions again, in the same order, carrying a running count `k` of how many non-zeros have been written so far. At the **start** of each row, write the current `k` into `m_row_index`; then for each non-zero member in the row, append its value to `m_values` and its column to `m_col_index` and advance `k`.

AT	WRITE	NON-ZEROS FOUND IN THIS ROW	<code>k</code> AFTER
start of row 0	<code>m_row_index[0] = 0</code>	10 at col 0, 20 at col 1	2
start of row 1	<code>m_row_index[1] = 2</code>	30 at col 1, 40 at col 3	4
start of row 2	<code>m_row_index[2] = 4</code>	50 at col 2, 60 at col 3, 70 at col 4	7
start of row 3	<code>m_row_index[3] = 7</code>	80 at col 5	8
after last row	<code>m_row_index[4] = 8</code>	—	8

The final write is not part of the loop and is the one most often forgotten. It is what gives `m_row_index` its $m + 1$ entries, and it is why the last entry always equals `m_nonzeros`.

2.2.4 The finished arrays

`m_values` and `m_col_index` are one list read two ways: slot k holds a value and the column it came from.

k	0	1	2	3	4	5	6	7
<code>m_values</code>	10	20	30	40	50	60	70	80
<code>m_col_index</code>	0	1	1	3	2	3	4	5
belongs to	row 0		row 1		row 2		row 3	

Notice that neither array records a row. That last line of the table is not stored anywhere — it is **derived**, and `m_row_index` is what derives it, by giving the boundaries rather than labelling each slot.

i	<code>M_ROW_INDEX[i]</code>	SLICE OF ROW i	NON-ZEROS IN ROW i
0	0	$[0, 2)$	2
1	2	$[2, 4)$	2
2	4	$[4, 7)$	3
3	7	$[7, 8)$	1
4	8	terminator	—

Each row’s slice runs from its own entry to the **next** one, which is why an entry per row is not enough and the array needs $m + 1$. The per-row counts in the last column are just the differences between consecutive entries, and they sum to 8 — the same statement as “the last entry is `m_nonzeros`”.

※ **Remark** Nothing in the three arrays is mn -sized, and nothing scales with the zeros. Storage is $2 \times \text{nonzeros} + m + 1$ integers: here $2 \times 8 + 5 = 21$ against the dense 24. The saving looks thin at this size because the matrix is small and only 66% sparse — the format is built for the case where $\text{nonzeros} \ll mn$, and it is $m + 1$, never $m \times n$, that the overhead tracks.

2.2.5 Three variants worth tracing

A row with no non-zeros. The second driver matrix, $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 100 & 200 \\ 0 & 0 & 300 \end{bmatrix}$, compresses to

```
m_values    = [ 100  200  300 ]
m_col_index = [   1    2    2 ]
m_row_index = [   0    0    2    3 ]
```

Row 0 contributes nothing, so `k` is still 0 when row 1 starts, and `m_row_index[0]` and `m_row_index[1]` are both 0. A repeated entry is row 0 saying “my slice is empty” — $[0, 0)$ contains nothing. This is not a bug and not a sentinel; it is the general rule producing the right answer with no special case. An implementation that only writes `m_row_index[i]` when it finds something in row i breaks here.

Fewer data than the matrix. `compress(4, 6, array1, 10)` supplies only flat indices 0..9. Rows 0 and 1 are covered (row 1 ends at index 11, but its two missing members were zeros anyway); rows 2 and 3 are entirely absent and are therefore all zeros. So:

```
m_values    = [ 10  20  30  40 ]
m_col_index = [  0   1   1   3 ]
m_row_index = [  0   2   4   4   4 ]
```

The repeated tail is the same phenomenon at the other end — two empty rows in a row. The object is still a valid 4×6 matrix; `getAt(3, 5)` returns 0 rather than throwing, because row 3 exists and its member is zero.

A 0×0 request. `compress(0, 0, array1, 24)` ignores the data entirely and yields an empty object: all three pointers null, `dump()` prints `The object is empty!`. Contrast with `compress(3, 3, array1, 0)`, which yields a real 3×3 matrix of zeros — `m_row_index = [0 0 0 0]`, four entries, allocated. See [the comparison below](#).

2.2.6 Repeats, and which ones are legal

Nothing so far has repeated a **value**, which makes the driver's examples slightly misleading. This one repeats almost everything that can repeat:

$$\begin{bmatrix} 7 & 0 & 7 & 0 \\ 0 & 0 & 0 & 0 \\ 7 & 0 & 0 & 5 \end{bmatrix} \quad (2.2)$$

```
m_values    = [ 7  7  7  5 ]
m_col_index = [ 0  2  0  3 ]
m_row_index = [ 0  2  2  4 ]
```

Three different kinds of repetition are visible at once, and they mean three different things:

REPEAT	WHAT IT MEANS
7 three times in <code>m_values</code>	Nothing. CSR stores values per position, not as a set. A 3×4 matrix of all 7s would give twelve of them.
Column 0 twice in <code>m_col_index</code>	Nothing — the two are in different slices, $k = 0$ in row 0 and $k = 2$ in row 2. Column numbering restarts with every row.
2 twice in <code>m_row_index</code>	Row 1 is empty: its slice is $[2, 2)$, which contains nothing.

What is **not** there is any 0 in `m_values`, and there never can be — that is invariant 3, and it holds no matter how many zeros the matrix has.

And within any one slice, the column indices strictly increase:

ROW	SLICE	COLUMNS IN IT	STRICTLY INCREASING?
0	$[0, 2)$	0, 2	yes
1	$[2, 2)$	—	vacuously — the slice is empty
2	$[2, 4)$	0, 3	yes

That is not a coincidence to be checked for; it falls out of scanning each row left to right and storing every position at most once. It is the property `getAt`'s linear scan quietly depends on, and it is what would let that scan be replaced by a binary search on a large matrix.

Reading values back shows why the duplicates are harmless:

CALL	SLICE	SCAN	RESULT
<code>getAt(0, 2)</code>	$[0, 2)$	col 0, then col 2 — match at $k = 1$	<code>m_values[1] = 7</code>
<code>getAt(2, 0)</code>	$[2, 4)$	col 0 — match at $k = 2$	<code>m_values[2] = 7</code>
<code>getAt(0, 1)</code>	$[0, 2)$	cols 0, 2 — no 1	0
<code>getAt(1, 0)</code>	$[2, 2)$	empty range, loop never runs	0

Two lookups return 7 from two different slots, and neither one has to know the other exists. The scan matches on **column**, never on value, so equal values are simply unrelated data that happen to coincide.

◆ **COMMON PITFALL** The one repeat that is always a bug is a column index appearing twice **inside the same slice** — `m_col_index = [0 0 0 3]` here. It means one (r, c) position was written twice. `getAt` would return whichever copy came first, `m_nonzeros` would overcount, the row's slice would be one entry too wide and push every later row's offset out by one, and `operator==` would call this matrix unequal to a correctly compressed copy of itself.

It is worth a test: for each row, walk the slice and assert the columns strictly increase.

2.3 Reading a member back

⌘ Formula 2.1 (Random access)

```
row_start = ROW_INDEX[row]
row_end   = ROW_INDEX[row + 1]
```

Scan `k` over `[row_start, row_end)`. If `COL_INDEX[k] == col` the answer is `v[k]`; if the scan finishes without a match, the member is zero.

This is why `m_row_index` needs that extra $(m + 1)$ -th entry — the last row would otherwise have a start and no end.

Against the 4×6 arrays above:

CALL	SLICE	SCAN	RESULT
<code>getAt(0, 1)</code>	<code>[0, 2)</code>	<code>m_col_index[0]</code> is 0, no; <code>[1]</code> is 1, match	<code>m_values[1] = 20</code>
<code>getAt(2, 4)</code>	<code>[4, 7)</code>	cols 2, 3, then 4 — match at $k = 6$	<code>m_values[6] = 70</code>
<code>getAt(2, 5)</code>	<code>[4, 7)</code>	cols 2, 3, 4 — no 5	0, a real member
<code>getAt(1, 0)</code>	<code>[2, 4)</code>	cols 1, 3 — no 0	0, a real member
<code>getAt(3, 5)</code>	<code>[7, 8)</code>	col 5 — match at $k = 7$	<code>m_values[7] = 80</code>
<code>getAt(4, 0)</code>	—	row 4 is not less than <code>m_m</code> , which is 4	throws
<code>getAt(0, 6)</code>	—	col 6 is not less than <code>m_n</code> , which is 6	throws

The second row of that table is the driver's `acSRList.getAt(1, 2, 4)`: list index 1 is the 4×6 matrix, and $(2, 4)$ is 70, which is what `driver.txt` prints.

Rows three and four are the distinction the whole design turns on. A zero that was never stored is still a member of the matrix and still has an answer. Only an index that does not exist is an error.

On the 3×3 matrix, `getAt(0, 0)` is the degenerate case: `row_start` and `row_end` are both 0, the scan range is empty, the loop body never runs, and the function returns 0 without ever touching `m_values`. Any lookup in an empty row costs nothing.

◆ **WHY IT WORKS** Access is $O(k)$ in the number of non-zeros **in that row**, not $O(1)$. That is the trade the format makes: a dense array gives constant-time access and pays $m \times n$ in memory; CSR gives near-constant access on sparse data and pays only for what is actually there.

2.4 Sparsity

The sparsity ratio is the share of members that are zero, reported as a truncated integer percentage:

$$\text{sparseRatio} = \left\lfloor ((m \times n) - \text{nonzeros}) \times \frac{100}{m \times n} \right\rfloor \quad (2.3)$$

A ratio of 0.656 reports as 65, not 66. The example above has 8 non-zeros out of 24 members, so $16 \times 100/24 = 66$.

3 Function-by-function intent

3.1 CSR

FUNCTION	WHAT IT IS FOR
<code>CSR()</code>	Produce an empty object holding no memory. Every member initialised, all three pointers null.
<code>~CSR()</code>	Free the three arrays. Never <code>m_next</code> .
<code>CSR(const CSR&)</code>	Deep copy: new arrays, same contents, independent lifetime. <code>m_next</code> is reset to null, not copied.
<code>compress(...)</code>	The only allocating function. Turns a row-major array into the three CSR arrays, re-callable on a live object.
<code>getAt(r, c)</code>	Random access by the rule above. Throws <code>runtime_error</code> when the indices do not exist.
<code>sparseRatio()</code>	Zero-share as an integer percentage. 0 if empty, 100 if every member is zero.
<code>operator=</code>	Same matrix — same shape, same members.
<code>clear()</code>	Back to the empty state, reusable.
<code>empty()</code>	Whether the object holds a matrix at all.
<code>dump()</code>	Given. Debug print of the three arrays.

3.2 CSRList

FUNCTION	WHAT IT IS FOR
<code>CSRList()</code>	Empty list: null head, size 0.
<code>~CSRList()</code>	Free every node.
<code>CSRList(const CSRList&)</code>	Deep copy preserving order .
<code>operator=</code>	Deep copy onto an existing list, self-assignment safe.
<code>operator==</code>	Same matrices in the same order.

FUNCTION	WHAT IT IS FOR
<code>insertAtHead(m)</code>	Push a copy of <code>m</code> onto the front.
<code>getAt(i, r, c)</code>	Access member (r, c) of the i -th matrix, head being index 0. Throws when i is not a position in the list.
<code>averageSparseRatio()</code>	Mean of the nodes' ratios.
<code>clear()</code>	Delete every node, back to empty.
<code>empty()</code>	Whether the list has any nodes.
<code>dump()</code>	Given. Debug print of every node, head first.

4 The invariants everything rests on

◆ Key idea (The contract)

1. `m_row_index` has exactly `m_m + 1` entries; `m_values` and `m_col_index` have exactly `m_nonzeros`.
2. `m_row_index[0] = 0` and `m_row_index[m_m] = m_nonzeros`, and the sequence never decreases.
3. **No explicit zero is ever stored.** `compress()` skips zeros, so nothing else may introduce one.
4. An empty object owns nothing: three null pointers, three zeroed counts.
5. A `CSR` owns its three arrays and nothing else. `m_next` is a link, not a child.
6. A `CSRList` owns every node reachable from `m_head`, and each node is owned by exactly one list.
7. Within one row's slice, `m_col_index` strictly increases. Across slices it may repeat freely, and so may `m_values`.

Invariant 3 is the quiet one, and it earns its keep in `operator==`. Because zeros are never stored, the compressed form is **canonical**: two equal matrices of the same shape always produce byte-identical arrays. That is what makes a straight elementwise comparison of `m_values`, `m_col_index` and `m_row_index` a correct test of matrix equality, with no decompression needed. Store one explicit zero anywhere and the comparison silently stops being valid.

Invariant 7 is the one the linear scan in `getAt` leans on without saying so, and it is the only kind of repetition in the whole structure that signals a bug — see [the repeats example](#).

Invariants 5 and 6 are the ownership model, and they are the whole reason `~CSR()` must leave `m_next` alone. If the destructor followed the link, then deleting the head would recursively destroy the list, and `CSRList::clear()` would then walk into freed memory and delete each node a second time.

5 Empty is not all-zero

The single most common misreading of this specification.

	EMPTY OBJECT	3 × 3 ALL-ZERO MATRIX
How it arises	<code>CSR()</code> , <code>clear()</code> , or <code>compress</code> with m or n zero	<code>compress(3, 3, arr, 0)</code>

	EMPTY OBJECT	3 × 3 ALL-ZERO MATRIX
<code>m_m</code> , <code>m_n</code>	0, 0	3, 3
<code>m_nonzeros</code>	0	0
<code>m_row_index</code>	<code>nullptr</code>	allocated, <code>[0 0 0 0]</code>
<code>empty()</code>	<code>true</code>	<code>false</code>
<code>sparseRatio()</code>	0	100
<code>getAt(0,0)</code>	throws	returns 0

◆ **COMMON PITFALL** Testing `empty()` as `m_values == nullptr` puts the second column on the wrong side of every row in this table: an all-zero matrix has `m_nonzeros == 0`, so `m_values` is legitimately empty while the object is not. Key `empty()` off `m_row_index`, which a real matrix always allocates, or off the dimensions.

6 Input handling in `compress`

The “Additional Requirements” section is really a specification of one function. Collected:

INPUT	REQUIRED BEHAVIOUR
<code>arraySize < mn</code>	Use what is supplied; the remaining matrix members are zero.
<code>arraySize > mn</code>	Use the first mn entries; ignore the tail.
$m = 0$ or $n = 0$	An empty object, even if data is supplied .
No data supplied	A genuine $m \times n$ matrix of all zeros — not an empty object.
Called a second time	Release the previous arrays first, then build afresh.

Row-major means index $in + j$ holds member (i, j) : for a 2×4 matrix, indices 0..3 are row 0 and 4..7 are row 1.

※ **Remark** A two-pass implementation — count the non-zeros, then allocate exactly, then fill — is cleaner than one pass with worst-case mn allocation, and it makes invariant 1 true by construction rather than by luck.

7 Memory, and where it goes wrong

The four failure modes this project is actually testing for:

FAILURE	HOW IT HAPPENS
Leak on recompress	<code>compress()</code> may be called again on a live object. Not releasing the old arrays first leaks all three.
Cascading delete	<code>~CSR()</code> deleting <code>m_next</code> destroys the rest of the list; <code>CSRList::clear()</code> then frees the same nodes a second time.

FAILURE	HOW IT HAPPENS
Aliased node	The copy constructor copying <code>m_next</code> , or <code>insertAtHead</code> linking the caller's object instead of a copy, leaves two owners for one node — or a list pointing at a stack object that has already died.
Self-assignment	<code>operator=</code> that clears before checking <code>this == &rhs</code> destroys the source it is about to read.

Three smaller ones worth naming:

- After `delete[]`, set the pointer to `nullptr`. That is what makes `clear()` idempotent and the destructor safe after an explicit `clear()`.
- In `CSRList::clear()`, save `temp→m_next` **before** `delete temp`. Reading through a freed pointer is a use-after-free even when it appears to work.
- In `sparseRatio()`, multiply before dividing. `zeros / total * 100` in integer arithmetic is 0 for every matrix that is not entirely zeros.

8 Order, and the two places it matters

`insertAtHead` means the **last** matrix inserted is at index 0. In `driver.cpp`:

```
aCSRList.insertAtHead(aCSR); // the 4x6 matrix → ends up at index 1
aCSRList.insertAtHead(cCSR); // the 3x3 matrix → ends up at index 0
```

which is why `driver.txt` prints the 3×3 matrix first, and why `aCSRList.getAt(1, 2, 4)` reaches into the 4×6 matrix and returns 70.

The copy constructor and `operator=` have to preserve that order exactly. The natural implementation — walk `rhs`, `insertAtHead` each node — **reverses** the list, and the specification calls this out by name. Either keep a tail pointer and append, or recurse to the end of `rhs` and build back to front.

- 🕒 **Observation** A reversed copy is invisible to most casual testing: the list still has the right size, the right matrices, and no memory errors. It shows up only in `operator=` against the original, or in a `getAt` at a specific index. That makes it a good candidate for an early, deliberate test.

9 Error signalling

Both `getAt` functions throw `runtime_error` from `<stdexcept>`, and the distinction they draw is the point:

- An index **outside** the matrix, or a `CSRIndex` that is not a position in the list, is an **error** — throw.
- An index **inside** the matrix whose member happens to be zero is a **valid answer** — return 0. The zero was never stored, but it is still a real member of the matrix.

`CSRList::getAt` should validate only the list index and then delegate; the row and column bounds are `CSR::getAt`'s business, and letting its exception propagate avoids two copies of the same check drifting apart.

`driver.txt` fixes the wording for the list case:

```
Exception Error: Object is not in the list!
```

10 Testing

`mytest.cpp` holds the `Tester` class, its implementation, every test case, and `main()`. Each test case is a separate function returning `bool`; visual inspection is not a result, and `dump()` must not be called from a test.

TARGET	CASE
<code>compress</code>	Normal: members populated as expected for a known array.
<code>compress</code>	Error: fewer data than <i>mn</i> .
<code>compress</code>	Error: 0×0 requested but data supplied.
<code>CSR::operator==</code>	Normal case.
<code>CSR::operator==</code>	Edge: both objects empty.
<code>CSR::getAt</code>	Error: indices out of range throw.
<code>CSRList::operator=</code>	Normal case.
<code>CSRList::operator=</code>	Edge: empty assigned onto a populated list.
<code>CSRList::getAt</code>	Error: the list is empty.
<code>CSRList::getAt</code>	Normal: returns the expected value.
everything	<code>valgrind</code> : no leaks, no errors.

That list is the required minimum, not a sufficient suite. The gaps worth filling, given the invariants above:

- Copy constructor and `operator=` preserve **order**, checked by index rather than by size.
- Self-assignment (`list = list`) leaves the list intact and leaks nothing.
- `compress()` called twice on the same object gives the second matrix, with no leak.
- `sparseRatio()` at both ends: empty $\rightarrow 0$, all-zero $\rightarrow 100$.
- `getAt` returns 0 for an in-range member that was never stored.
- A matrix with repeated values and repeated column indices compresses correctly, and no column index repeats inside a single row's slice.

※ **Remark** Build with `-g` and run under `valgrind -s --track-origins=yes` whenever anything looks wrong. Most of the failure modes in this project are silent under normal execution and loud under `valgrind`.

11 Suggested order of work

Each step is testable before the next one starts, which keeps any new bug inside the handful of lines just written.

1. `CSR()`, `clear()`, `empty()`, `~CSR()` — the empty state and its lifetime. Construct, destruct, and clear twice under `valgrind`.
2. `compress()` — check against the worked example above with `dump()`.
3. `getAt()` and its exception.
4. `sparseRatio()`, then `CSR::operator=`.
5. `CSR` copy constructor — verify `m_next` is null and the arrays are independent.

6. `CSRList()`, `empty()`, `insertAtHead()`, `clear()`, `~CSRList()` — then `dump()` should reproduce `driver.txt` 's ordering.
7. `CSRList::getAt()` and both exception paths.
8. Copy constructor, `operator=`, `operator==` for the list — the order-preservation problem.
9. `averageSparseRatio()` .
10. Run `driver.cpp` and diff against `driver.txt` ; run `mytest.cpp` under valgrind.